

OBTENCIÓN Y CREACIÓN DEL DATASET

DATASET 1: AudioSet

Es la referencia estándar de clasificación de audio, con millones de clips de 10 segundos extraídos de YouTube. Extraer audios de AudioSet no se puede realizar de forma directa, sino que requiere de un script ya que la base de datos no contiene archivos de sonido directos, sino un enorme archivo CSV con enlaces a vídeos de YouTube y marcas de tiempo temporales. Cabe destacar que algunos vídeos de YouTube de este dataset no están disponibles por una serie de razones, como por ejemplo que el autor lo haya borrado.

- Requisitos
 - pip install pandas yt-dlp
 - FFmpeg (motor de procesamiento de audio que yt-dlp necesita para recortar los clips y extraer el audio sin descargar los vídeos enteros) (winget install ffmpeg)
- Paso 1: Descarga metadatos (<https://research.google.com/audioset/download.html>)
 - descarga del archivo *balanced_train_segments.csv* (contiene una mezcla equilibrada de clases, ideal para entrenar modelos de inteligencia artificial sin sesgos masivos).
 - descarga del archivo *eval_segments.csv* (para validar el modelo más adelante)
- Paso 2: Ontology ID (<https://github.com/audioset/ontology/blob/master/ontology.json>)
 - **Shout:** /m/07p6fty
 - **Yell:** /m/07sr1lc
 - **Screaming:** /m/03qc9zr
 - **Crying, sobbing:** /m/0463cq4
 - **Baby cry, infant cry:** /t/dd00002
 - **Wheeze:** /m/07mzm6
 - **Snoring:** /m/01d3sd
 - **Cough:** /m/01b_21
 - **Sneeze:** /m/01hsr_
 - **Thump, thud:** /m/07qnq_y
 - **Bang:** /m/07pws3f
 - **Silence:** /m/028v0c
 - **Speech:** /m/09x0r
 - **Whispering:** /m/02rtxlq
 - **Laughter:** /m/01j3sz
 - **Breathing:** /m/0lyf6
 - **Walk, footsteps:** /m/07pbtc8
 - **Burping, eructation:** /m/03q5_w
 - **Chatter:** /m/07rkbfh
 - **Thunderstorm:** /m/0jb2l
 - **Water:** /m/0838f
 - **Vehicle horn, car horn, honking:** /m/0912c9
 - **Car passing by:** /t/dd00134
 - **Emergency vehicle:** /m/03j1ly
 - **Traffic noise, roadway noise:** /m/0btp2
 - **Door:** /m/02dgv
 - **Alarm clock:** /m/046dlr

- **Inside, small room:** /t/dd00125
- Paso 3: Script de filtrado y descarga
 - Fichero *audioSet_extractor.py*
- Resultados: 5465 muestras para el conjunto train y 4998 muestras para el conjunto eval.

DATASET 2: ESC-50

A diferencia de AudioSet, ESC-50 es mucho más ligero y directo, ideal para obtener muestras muy limpias de los episodios de tos, ronquidos o lloros que el sistema necesita clasificar. La versión oficial agrupa sus 2000 audios de 5s en una única carpeta, así que usaremos un pequeño script para extraer los que necesitamos basándonos en su código interno. Contiene 2000 audios en formato .wav (5s, 44.1kHz, mono) con el siguiente formato en cuanto a los nombres de los archivos: {FOLD}-{CLIP_ID}-{TAKE}-{TARGET}.wav

- {FOLD} - index of the cross-validation fold
- {CLIP_ID} - ID of the original Freesound clip,
- {TAKE} - letter disambiguating between different fragments from the same Freesound clip,
- {TARGET} - class in numeric format [0, 49].
 - Paso 1: Descarga del dataset y metadatos (<https://github.com/karolpiczak/esc-50>)
 - Metadatos: *esc50.csv*
 - Paso 2: Ontology ID
 - **Coughing:** 24
 - **Crying baby:** 20
 - **Sneezing:** 21
 - **Snoring:** 28
 - **Rain:** 10
 - **Toilet flush:** 18
 - **Thunderstorm:** 19
 - **Breathing:** 23
 - **Footsteps:** 25
 - **Laughing:** 26
 - **Clock alarm:** 37
 - **Siren:** 42
 - Paso 3: Script de filtrado
 - Fichero *esc50_extractor.py*
 - Resultados: 480 muestras.

DATASET 3: FSD50K

FSD50K es un dataset masivo alojado en la plataforma científica Zenodo e impulsado por la Universitat Pompeu Fabra. Su estructura requiere cruzar los nombres numéricos de los archivos de audio con un archivo CSV de metadatos para aislar las incidencias a clasificar.

- Paso 1: Descarga del dataset y metadatos (<https://zenodo.org/records/4060432>)
 - Metadatos: *FSD50K.ground_truth.zip*
- Paso 2: Ontology ID
 - **Burping and eructation:** /m/03q5_w
 - **Cough:** /m/01b_21

- **Crying and sobbing:** /m/0463cq4
- **Respiratory sounds:** /m/09hlz4
- **Screaming:** /m/03qc9zr
- **Shout:** /m/07p6fty
- **Sneeze:** /m/01hsr_
- **Thump and thud:** /m/07qnq_y
- **Yell:** /m/07sr1lc
- **Alarm:** /m/07pp_mv
- **Breathing:** /m/0lyf6
- **Car passing by:** /t/dd00134
- **Chatter:** /m/07rkbfh
- **Conversation:** /m/01h8n0
- **Door:** /m/02dgv
- **Human voice:** /m/09l8g
- **Laughter:** /m/01j3sz
- **Rain:** /m/06mb1
- **Siren:** /m/03kmc9
- **Speech:** /m/09x0r
- **Thunderstorm:** /m/0jb2l
- **Traffic and roadway noise:** /m/0btp2
- **Vehicle horn and car horn and honking:** /m/0912c9
- **Walk and footsteps:** /m/07pbtc8
- **Water:** /m/0838f
- Paso 3: Script de filtrado
 - Fichero `fsd50k_extractor.py`
- Resultados: 3950 muestras del conjunto eval y 9692 muestras del conjunto dev.

DATASET 4: COUGHVID

El dataset COUGHVID es un dataset de salud masivo creado durante la pandemia, repleto de muestras de toses aportadas por todo el mundo. Es la mejor fuente para entrenar el modelo en la detección de la incidencia de *tos persistente*. Como la base de datos contiene más de 20000 grabaciones y muchas tienen ruido o silencios, filtraremos estrictamente aquellas que el propio dataset certifica que contienen tos clara. El archivo CSV contiene una columna llamada *cough_detected*. Es un valor del 0 al 1 que indica la probabilidad de que el audio contenga tos real. Filtraremos solo aquellos con un valor superior o igual a 0.95 (95% de confianza).

- Paso 1: Descarga del dataset y metadatos (<https://zenodo.org/records/4048312>)
 - Metadatos: *metadata_compiled.csv*
- Paso 2: Criterios de selección
 - **Cough detected:** ≥ 0.95
 - **quality_1 o quality_2 o quality_3:** {ok, good} o vacía
 - **severity_1 o severity_2 o severity_3:** {severe} o vacía
- Paso 3: Script de filtrado
 - Fichero *coughvid_extractor.py*
- Resultados: 5902 muestras.

TOTAL: 30487 muestras

FORMATO DE LOS AUDIOS

Se usará el Formato 16 kHz mono, ya que es el estándar TinyML. La frecuencia de muestreo (16 kHz) significa que el audio captura 16000 puntos de datos por segundo. Según el teorema de Nyquist, esto permite registrar frecuencias de hasta 8 kHz. Como la voz humana, los ronquidos y los golpes concentran casi toda su información acústica por debajo de los 4 kHz, no perdemos información; usar 44.1 kHz (calidad CD) solo añadiría frecuencias agudas inútiles y triplicaría el peso del archivo, colapsando la memoria RAM del microcontrolador durante la generación del espectrograma. El formato mono indica que el audio solo tiene 1 canal. Los micrófonos estéreo generan dos pistas de audio simultáneas (izquierda y derecha), pero el modelo no necesita saber de qué lado del dispositivo proviene el sonido, solo si ha ocurrido y cómo ha ocurrido. El formato mono divide el procesamiento matemático a la mitad.

Los scripts que obtienen audios de AudioSet y COUGHVID ya integran FFmpeg para forzar el formato `-ar 16000 -ac 1` durante la extracción. Sin embargo, ESC-50 y FSD50K realizan una copia directa del .wav original usando `shutil.copy2`. Para estandarizar estos dos últimos y unificar todo el dataset, se ejecuta el script *UNIFICADOR.py*.

De hecho, se ejecuta también el script que aplica la función FFmpeg a los audios de los dos primeros datasets por los motivos que siguen:

- Sin pérdida de calidad: Como los archivos .wav utilizan formato PCM (audio sin compresión), recodificar un audio que ya está a 16 kHz mono bajo esos mismos parámetros es un proceso transparente que no degrada el sonido, a diferencia de lo que ocurriría con formatos comprimidos como MP3.
- Limpieza de metadatos: FFmpeg reescribirá desde cero la cabecera técnica del archivo WAV. Esto es muy beneficioso para unificar el dataset, ya que elimina metadatos residuales, etiquetas extrañas o codificaciones anómalas que pudieran venir de los datasets originales y que a veces generan errores de lectura al subir los datos a Edge Impulse.
- Impacto mínimo en el sistema: El único efecto real es que el procesador invertirá unas milésimas de segundo extra en leer y reescribir un archivo que ya tenía el formato correcto, un tiempo totalmente despreciable.